

Automatización LowCode en AetherNet: Notificación de Intrusión vía Telegram Bot API Directo (Node-RED en Deuda Técnica) – HU-02

Andrés Felipe Martínez Henao
Automatización y LowCode – Sin lineamientos (provisional)
Tecnología en Desarrollo de Software (5º sem) – Ing. Sistemas y Computación (6º sem)
Universidad Tecnológica de Pereira
F.martinez5@utp.edu.co

Proyecto AetherNet – Área 3
IDs LOW-01..07 – docs/Automatizacion-LowCode/ – RF-4.1/HU-02

Abstract—Esta materia no cuenta aún con lineamientos docentes formales; el artículo se basa en los roadmaps/backlogs internos y en las decisiones ADR-001 y deuda LOW-02. El alcance original incluía bombillo Tuya (tuya-local, RF-4.2) y Node-RED como motor LowCode (RF-4.1). Ambas capas fueron reconducidas: Tuya se cancela definitivamente el 2026-09-01 (ADR-001, R-01: exigir cuenta propietaria y local_key viola RNF-3.1 FOSS) y Node-RED pasa a deuda técnica el 2026-09-09 (indicación asesor: no deploy esta iteración). El flujo vigente HU-02 se resuelve sin Node-RED: MEGA KY-008→UART SECURITY:→Gateway ESP32→MQTT aethernet/seguridad/intrusion:72→Telegram Bot API directo (POST https://api.telegram.org/bot<token>/sendMessage, parse_mode Markdown) + LED RGB rojo 3 s + App dashboard. El flujo JSON automation/flows/intrusion_alert.json (mqtt-intrusion→function-parse-intrusion→telegram-alert→http-telegram) queda como referencia exportable de 7 nodos para retomar en Sprint 4. Se documentan topics aethernet/# (DEVOPS-02), endpoint POST /api/security-events, manejo de reconexión y criterios E2E (latencia <3 s, 0 falsos positivos en 10 disparos). Estado: LOW-01/04 Won't, LOW-02 deuda, pendiente LOW-03 bot directo y LOW-05 E2E Sprint 4.

Index Terms—LowCode, Node-RED deuda técnica, Telegram Bot API, MQTT aethernet/seguridad/intrusion, HU-02, ADR-001, FOSS

I. INTRODUCCIÓN: LOWCODE PROVISIONAL

A diferencia de TS4D3/TS683/TS6C3/DevOps, Automatización LowCode no tiene PDF académico entregado por el docente al 2026-09-11. El contenido aquí es *provisional* y se deriva de docs/Automatizacion-LowCode/roadmap-lowcode.md, backlog-lowcode.md, docs/roadmap.md §3/§8, docs/backlog.md Área 3 y la bitácora Diario_LowCode.ipynb. La evaluación se anticipa por HU-02 y RF-4.1/4.2, con trazabilidad a ADR-001 y risk R-01.

II. ALCANCE Y DECISIONES

A. Alcance PRD y RF

PRD §4 In: automatización LowCode con Node-RED + Telegram (asesor: flujo intrusion_alert.json exportable); RF-4.1 captura eventos críticos → Telegram; RF-4.2 (cancelado) bombillo Tuya. KPI HU-02: latencia intrusión→notificación y 0% falsos positivos láser (PRD §5).

B. ADR-001: cancelación Tuya (2026-09-01)

Barrera técnica (lote Mercury LB401 20MER93 sin local_key estable) + barrera determinante de políticas API: obtener local_key exige Tuya IoT Platform propietaria → viola RNF-3.1 FOSS y crea vendor lock-in. Decisión: RF-4.2→Won't, HU-02 sin bombillo, LOW-01/04→W, flujo JSON simplificado (se eliminan tuya-bulb-alert, tuya-local-send), dependencia node-red-contrib-tuya-local removida, R-01/R-07 cerrados. Verificación: grep -i tuya solo retorna líneas CANCELADO; JSON válido 7 nodos.

C. Deuda LOW-02: Node-RED no deployado (2026-09-09)

Indicación asesor: no usar Node-RED esta iteración. El flujo automation/flows/intrusion_alert.json (mqtt in aethernet/seguridad/intrusion → function-parse-intrusion → telegram-alert → http-telegram → debug) queda versionado como referencia para Sprint 4. HU-02 se cumple vía Telegram HTTP directo, sin capa LowCode en runtime (architecture.md §3/§5, sprints.md:42).

III. MÉTODO: FLUJO VIGENTE HU-02 SIN NODE-RED

A. Cadena E2E (Fig. 1)

1) KY-008 se interrumpe → MEGA laser.cpp:15 (50 ms CHECK, COOLDOWN 3 s) → 2) LED RGB rojo 3 s no bloqueante (feedback local sin red) → 3) UART SECURITY:{"event_type":"intrusion"}

Table I
TOPICS Y PAYLOADS (ARCHITECTURE.MD §7, ACL.CONF:1)

Topic	Dirección	Payload JSON
aethernet/seguridad/intrusion	MEGA→GW→Backend/App	SecurityEvent 72
aethernet/access/event	MEGA→GW→App	AccessEvent 73
aethernet/rover/telemetry	Rover→GW→App	RoverTelemetry 69
aethernet/rover/command	App→GW→Rover	RoverCommand 69
aethernet/system/status	Gateway heartbeat	status 73

Table II
BACKLOG LOWCODE (BACKLOG.MD ÁREA 3 + BACKLOG-LOWCODE.MD)

ID	Tarea	Prioridad	Estado
LOW-01	Validar tuya-local	W	Cancelado ADR-001
LOW-02	Flujo Node-RED sub	W	Deuda 2026-09-09 (ref JSON)
LOW-03	Bot Telegram directo	M S4	Pendiente
LOW-04	Integración tuya-local	W	Cancelado ADR-001
LOW-05	Intrusión Telegram+rojo	M S4	Pendiente E2E
LOW-06	Dashboard Node-RED	C S4	Opcional
LOW-07	Reconexión broker	W	Deuda

38400 bd → 4) Gateway publica
aethernet/seguridad/intrusion → 5)
Backend persiste POST /api/security-events
(PostgreSQL) → 6) Gateway/Backend dispara POST
https://api.telegram.org/bot<token>/sendMessage
con chat_id + parse_mode Markdown → 7) App
suscrita refleja alerta en dashboard (RF-1.1). Pasos 2 y 6 son
independientes (LED funciona sin MQTT/Telegram).

B. Telegram Bot directo (LOW-03)

Creación vía BotFather (/newbot → token + chat_id),
envío HTTP con Markdown, token solo en variable
entorno (process.env.TELEGRAM_TOKEN), nunca en
JSON versionado (grep token auditoría). Formato men-
saje: hora/sensor/acción sugerida, prioridad alta HU-02, <2 s
tras publicar intrusión (criterio LOW-03).

C. Node-RED como referencia

Importar JSON en runtime local (misma LAN),
credenciales broker usuario nodered (ACL DEVOPS-
02: read aethernet/seguridad|access/#),
function-parse-intrusion filtra duplicados (láser
puede disparar ráfaga) con debounce/rate-limit, debug
verifica parseo. Manejo caída broker: reconexión o pérdida
documentada (LOW-07). Dashboard Node-RED opcional
(LOW-06) reutiliza mismos topics si sobra tiempo.

IV. RESULTADOS Y ESTADO BACKLOG

Diario_LowCode.ipynb documenta estado con
diagrama Mermaid + flujo validado sin Node-RED
(MEGA→Gateway→MQTT→Telegram+LED+App). Evi-
dencia Sprint 3: láser v2 c7ce065 + T1_mqtt_live.log
con seguridad/intrusion 303 líneas.

V. DISCUSIÓN Y LIMITACIONES

La deuda Node-RED simplifica deploy LAN y preserva
FOSS, pero pospone la evidencia LowCode “pura” (flujo
ejecutándose). La limitación se mitiga entregando JSON im-
portable + bitácora + E2E Telegram directo medible. Riesgo:
si el docente exige demo Node-RED, el flujo está listo para
deploy en 1 sprint sin cambiar HU-02. Recomendación: acor-
dar criterios de evaluación provisional por escrito y registrar
este artículo como anexo.

VI. CONCLUSIONES

HU-02 es cumplible 100% FOSS sin Tuya ni Node-RED en
runtime; la trazabilidad ADR-001 + deuda LOW-02 + Tele-
gram directo satisface RF-4.1 con dos capas independientes
(LED determinista + Telegram).

REFERENCES

- [1] AetherNet, Roadmap LowCode, docs/Automatizacion-LowCode/roadmap-lowcode.md.
- [2] AetherNet, Backlog LowCode LOW-01..07, docs/Automatizacion-LowCode/backlog-lowcode.md.
- [3] AetherNet, Roadmap §3 Automatizaciones, docs/roadmap.md.
- [4] AetherNet, Backlog Área 3, docs/backlog.md.
- [5] AetherNet, Arquitectura §3/§5/§7 topics, docs/architecture.md.
- [6] AetherNet, ADR-001 Cancelación Tuya, docs/adr/adr-001-cancelacion-tuya.md.
- [7] AetherNet, automation/flows/intrusion_alert.json (7 nodos, referencia).
- [8] AetherNet, docs/Automatizacion-LowCode/notebook/Diario_LowCode.ipynb.
- [9] AetherNet, Sprints §4 + deuda LOW-02 2026-09-09, docs/sprints.md.
- [10] AetherNet, PRD §4 In/Out + §5 KPI 0% falsos positivos, docs/prd.md.